

1. RECOPIACIÓN DE PARÁMETROS Y MÉTRICAS DEL SISTEMA

Parámetros Configurables (Variables Independientes)

- **Topología de Red:** Número de clientes totales esperados por ronda (*clientes_esperados*).
- **Hiperparámetros de Entrenamiento Local:** Épocas locales (*epochs_locales*), tamaño del *batch*.
- **Distribución de Datos:** Cantidad de muestras por cliente (*num_muestras*) y grado de heterogeneidad de los datos (IID vs. Non-IID).
- **Eficiencia de Comunicación:** Nivel de precisión de los pesos (*Cuantización a Float32 vs Float16*) y algoritmo de compresión (*np.savez_compressed vs .npz estándar / .bin crudo*).
- **Naturaleza del Cliente:** Hardware real (*dispositivo Android*) vs. Simuladores lógicos (*scripts en Python*).

Métricas Recogidas (Variables Dependientes - Registradas en el CSV)

- **Rendimiento del Modelo:**
 - *Accuracy Global (%)*: Precisión del modelo tras la agregación en el servidor.
 - *Loss Global*: Función de pérdida que indica el margen de error del modelo.
 - *Velocidad de Convergencia*: Número de rondas necesarias para alcanzar un *Accuracy* objetivo (ej. 90%).
- **Rendimiento de Red y Hardware:**
 - *Tráfico Total por Ronda (KB)*: Suma del tamaño de subida y bajada de todos los clientes.
 - *Tiempo Medio de Entrenamiento Local (s)*: Coste computacional puro en los dispositivos.
 - *Tiempo Total de Ronda (s)*: Duración de punta a punta del ciclo completo.
 - *Efecto Straggler / Espera Rezagados (s)*: Tiempo que el servidor (o los clientes rápidos) pierde esperando al cliente más lento.

2. BATERÍA DETALLADA DE EXPERIMENTOS

BLOQUE 1: Hardware Real y Cuellos de Botella Físicos

Objetivo: Demostrar que la implementación Android + TFLite funciona y medir las limitaciones físicas y de red al combinar múltiples dispositivos reales de distintas capacidades comparadas con un PC.

- **Experimento 1.1: Validación funcional del cliente Android (Android Solitario)**
 - *Configuración:* 1 Servidor, 1 Cliente Android (100 muestras reales de MNIST). 50 rondas, 1 epoch local.
 - *Qué medir:* Validar funcionalmente el ciclo de vida de la aplicación Android. Demostrar que TFLite entrena correctamente on-device y que el empaquetado/desempaquetado de pesos en la red funciona sin errores.
- **Experimento 1.2: Flota Móvil Heterogénea (Diversidad de Hardware)**
 - *Configuración:* 1 Servidor + 3 Clientes Android reales de diferentes gamas.
Restricción estricta: Todos usarán exactamente la misma configuración, 100 muestras IID por cliente, 2 epochs locales y 30 rondas.
 - *Qué medir:* Aislamiento de la variable de cómputo móvil. Analizar la diferencia en el *Tiempo_Medio_Entrenamiento_Seg* de cada dispositivo según su procesador ARM. Demostrar el verdadero Efecto Straggler, registrando cuántos segundos (*Espera_Rezagados_Seg*) tiene que esperar el móvil más potente hasta que el dispositivo antiguo termina su entrenamiento para poder avanzar de ronda.

BLOQUE 2: Simulaciones a Escala y Eficiencia (Python)

Objetivo: Aprovechar la escalabilidad de los scripts Python para aislar variables matemáticas y de telecomunicaciones, comprobando el sistema bajo estrés masivo sin depender de disponer de decenas de teléfonos físicos.

- **Experimento 2.1: Distribución IID vs Non-IID (El reto de los datos)**
 - *Configuración:* 10 clientes Python. 30 rondas. 1 epoch local. 1000 muestras por cliente.
 - *Casuística A (IID):* Datos balanceados. Cada cliente tiene imágenes mezcladas de todos los números (0 al 9).
 - *Casuística B (Non-IID):* Partición estricta por etiquetas (Pathological Non-IID). Al cliente 1 se le asignan exclusivamente etiquetas 0 y 1, al cliente 2 las 2 y 3, etc.
 - *Qué medir:* La curva de *Accuracy*. Demostrar cómo el modelo federado sufre turbulencias y tarda más rondas en aprender cuando los datos de los usuarios son radicalmente distintos entre sí (Non-IID).
- **Experimento 2.2: Computación vs. Comunicación (Hiperparámetros)**
 - *Configuración:* 5 clientes Python (IID). Objetivo del 92.5% de *Accuracy*. Límite estricto de 50 rondas máximas por si alguna configuración no alcanza el objetivo. Variar epochs locales entre 1, 5 y 10. 1000 muestras cada cliente
 - *Qué medir:* Encontrar el punto perfecto. Se medirá el *Tráfico_Ronda_KB*, el tráfico acumulado y el tiempo total necesario para alcanzar un determinado nivel de precisión (un accuracy del 92.5%).
- **Experimento 2.3: Telecomunicaciones Puras (Compresión y Cuantización)**
 - *Configuración:* 5 clientes Python. 20 rondas. 3 epochs locales. 1000 muestras por cliente.
 - Float32 sin compresión (np.savez).
 - Float32 con compresión (np.savez_compressed).
 - Float16 sin compresión.
 - Float16 con compresión.

- *Qué medir:* Comparar la columna *Trafico_Ronda_KB*. Demostrar el porcentaje exacto de ancho de banda ahorrado mediante estas dos técnicas (vital para conexiones móviles 3G/4G) y comprobar si el *Accuracy* sufre alguna caída apreciable por la pérdida de precisión de los 16 bits.
- **Experimento 2.4: Escalabilidad respecto al número de clientes**
 - *Configuración:* Volumen total de datos constante (ej. 10.000 muestras). 1 epoch local y límite de 20 rondas.
 - *Casuística:* Repartir ese volumen fijo entre 2, 5, 10 y 20 clientes simulados.

Qué medir: Cómo afecta el número de participantes al tiempo de ronda y al coste de comunicación.

BLOQUE 3: Entornos Híbridos y Caóticos

Objetivo: Simular un entorno real donde miles de dispositivos diferentes colaboran, algunos entran, algunos tienen muchos datos y otros pocos.

- **Experimento 3.1: Integración equilibrada de clientes Android y Python**
 - *Configuración:* Entorno mixto. 3 Clientes Android + 2 Clientes Python. 20 rondas con 1 epoch local.
 - *Casuística:* Utilizar el escenario de datos asignados 1:1 (100 muestras tanto clientes Python como Android) para validar el funcionamiento de una flota híbrida con igual número de muestras por cliente.
- **Experimento 3.2: Agregación Desbalanceada (Pesos Ponderados)**
 - *Configuración:* Entorno mixto. 3 Clientes Android + 2 Clientes Python. 20 rondas con 1 epoch local.
 - *Casuística:* Modificar la proporción de datos asignados a Python frente a las 100 muestras fijas de Android. Se probarán los escenarios 1:10 (Python 1.000) y 1:50 (Python 5.000).
 - *Qué medir:* Cómo la matemática del FedAvg ponderado asimila a los clientes menores sin romper la convergencia.